Cosmetics E-Commerce Sales & Performance Analytics

Fikret Bartu Hıraekşi

Tools used:

- Python 3
- Jupyter Notebook
- Dbeaver
- PowerBI
- Claude (For creating the synthetic mock data.)

Libraries used:

Pandas (Data manipulation)

Introduction

Power BI is a great tool for businesses to visualize their data, create interactive dashboards, and gain insights into their overall performance. To use this tool to its full capacity, businesses should have clean data free of inconsistencies, critical missing values, and misleading entries. Additionally, while source transactional databases should be normalized according to well-known normal forms such as 1NF, 2NF, and 3NF to prevent data redundancy and anomalies, analytical data warehouses often denormalize this data into dimensional models. While related information from the normalized database is organized into descriptive tables (referred to as dimension tables), critical business metrics from those processes are aggregated into the 'fact table'. Dimension tables contain qualitative information such as descriptive texts, categories, or attributes, whereas fact tables contain more comprehensive numerical measurements, metrics, and facts about business operations. The dimension tables surrounding the fact table create schemas like star schema, snowflake schema, or galaxy schema. The schemas derived from data housed in a data warehouse can significantly boost query performance and make the database much easier for users to interpret and query. I will use many of the concepts I mentioned in this introduction and visualize a fake cosmetics e-retail store's business processes in an explainable and interactive way.

Understanding the Data

This data was created using Claude AI. It is synthetic data that mimics the operations of a cosmetics e-retail brand that has many stores across the United States. It consists of six different CSV files that are partially normalized. There are several issues with the data, such as ID inconsistencies, format errors, and incorrect data types. Before we fix those issues and create a comprehensive fact table, we have to understand what is inside them and how we can benefit from them.

Breakdown of the Tables

CUSTOMERS.CSV:

customer_id, first_name, last_name, gender, age_bracket, state, city, signup_date, acquisition_channel, loyalty_tier, lifetime_orders

It is the customer master list. Consists one row per customer. It records user information such as the first name, last name, that was used for sign-up, the loyalty tier, age. It answers the 'who' question about demographics.

PRODUCTS.CSV:

product_id, product_name, brand, category, subcategory, unit_cost, unit_price, launch_date, is_active

It is the product catalog consisting of the unique attributes of products that the store has been selling. There are 13 brands, 6 categories, and 22 subcategories that differentiate the products. It answers the 'what' question about the transaction.

ORDER_ITEMS.CSV:

order_item_id, order_id, product_id, quantity, unit_price, discount_pct, line_total

It stores individual product purchases within an order. Unlike the orders table, it records transactions by each distinct product ordered, offering a deeper breakdown of the orders table.

CAMPAIGNS.CSV:

campaign_id, campaign_name, campaign_type, start_date, end_date, discount_pct, channel_scope, target_category

It is the promotion calendar. Stores not only the dates, but also the information about if the campaign targets a specific category or which channel it applies to and the percent of discount.

STORES.CSV:

store_id, store_name, city, state, region, store_type, open_date

It is the physical data directory. It consists of the unique IDs of the stores, their names, and their relevant locations, such as the state and city where they are located. It answers the 'where' question. There are 28 stores within 12 states across the U.S., and an online store that delivers to all places in the country.

ORDERS.CSV:

order_id, order_date, customer_id, channel, store_id, payment_method, campaign_id, order_total

It serves as the transaction header for 52,389 individual purchases. It is the order-per-row unlike the *order_items.csv thus it contains the order_total. Also stores the information about who created the order, in which store, using what, during which campaign. It is a comprehensive record of transactions.*

Cleaning

The cleaning process was performed in a Jupyter Notebook using Python and the Pandas data manipulation library. Data cleaning is crucial for deriving meaningful insights from the data.

```
tables = {"customers":customers, "campaigns":campaigns, "order_items":order_items, "orders":orders, "products":products, "stores":stores}

for name, table in tables.items():
    print(f"Null Values For {name} Table")
    print(table.isnull().sum())
    print("-" * 30)
```

The first step is to detect the null values in the tables. Null values can be caused by several different reasons, thus they should be treated as separate cases, not solved by a single method.

- Null state information filled with ‘Unknown’.
- Null target category information filled with ‘General’.
- Store ID’s of Online purchases turned into ‘Online’, then orders with no values had been deleted.
- Null campaign id information filled with ‘No Campaign’.
- Null subcategory information filled with ‘Unknown’.

```
for name, table in tables.items():
    print(f"Duplicated Rows For {name} Table")
    print(table.duplicated().sum())
    print("-" * 30)
```

Then, duplicate rows were detected and removed from the tables.

When examining the data, the states are stored as abbreviations, such as "CA" for California and "GA" for Georgia. Since this information will be used in our Power BI

report, these abbreviations were transformed into their full state names. Additionally, leading and trailing spaces in some city names were cleaned up as well.

Like in the ‘city’ column, there were many entries with leading and trailing spaces.Also there were columns that In order to fix them;

- The dollar symbol and the leading and trailing spaces were removed in the unit price column.
- The leading and trailing spaces were removed and the capitalization has been corrected in the brand names.
- The leading and trailing spaces were removed and the capitalization has been corrected in the payment methods.

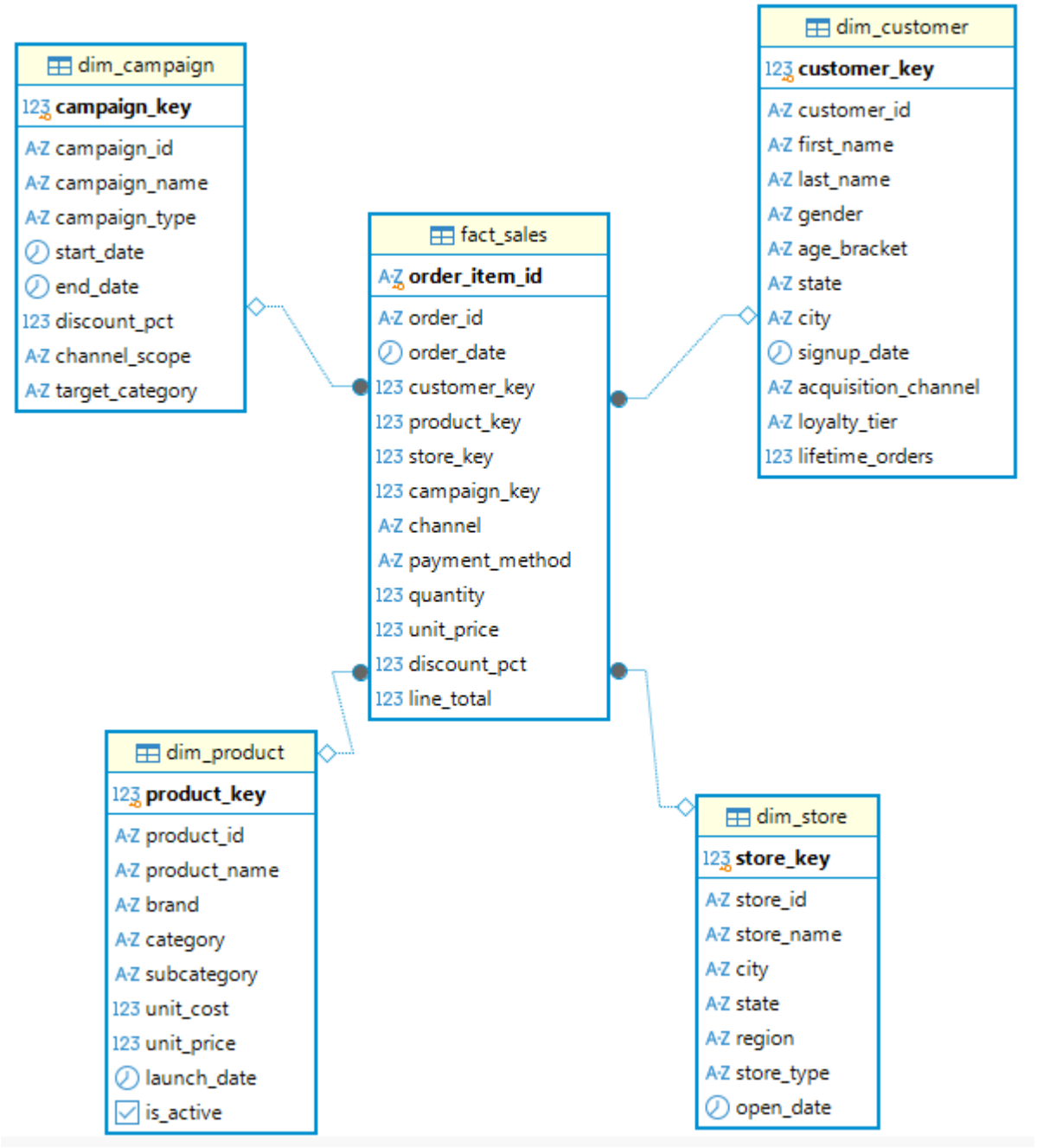
To utilize date columns effectively, they must be converted into the datetime data type. Therefore, all columns representing dates were transformed into datetime format.

The line_total column in the order_items table represents the amount spent on that specific line item, whereas the order_total column in the orders table represents the total amount spent on the entire order. To resolve any inconsistencies, the order_total column was updated to match the sum of the line_total values for each unique order_id.
Lastly, there were inconsistencies with product id’s in the order_items table. So any invalid product id’s filtered through the actual product id’s from the products table.

```
order_total_check = pd.merge(orders, order_items, on="order_id").groupby("order_id").agg({"line_total": "sum", "order_total": "min"})
order_total_check["line_total"] = order_total_check["line_total"].round(2)
order_total_check["order_total"] = order_total_check["order_total"].round(2),
orders["order_total"] = orders["order_id"].map(order_total_check["line_total"]).combine_first(orders["order_total"])
```

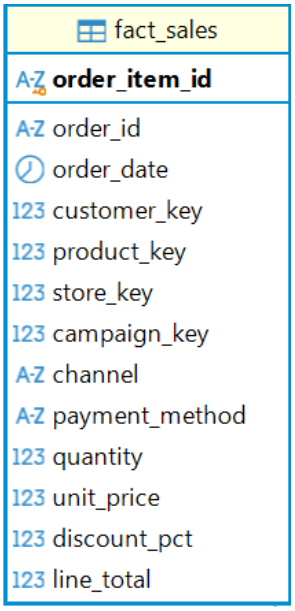
Creating the Fact Table

After the cleaning process, the table was transferred into the DBeaver. DBeaver is a popular database management tool. Also, PostgreSQL used the database engine in this project. In this process we'll create a fact table, then analyse some data to understand what we have better before doing the PowerBI report.



First of all, I created a namespace named Vaelora in the DBeaver, our imaginary name for the cosmetics brand. Then I wrote the script for schema design. In this new schema each table has an auto-generated integer surrogate key (generated always as identity) as its primary key. While the original ID columns were still kept.

As you can see the fact_table created at order-item grain, which displays transactions for each individual product ordered within a unique order.



Also the surrogate key values from dimension tables replaced the old ID values.

Because the fact table created using the joining the order table to order_items table.

A star-like design appeared in the diagram. Thus, this is called a ‘Star Schema’.

Querying the Data to Analyze

Customer LTV Value by Acquisition Channel and Loyalty Tier

```sql
with customer_ltv as(
    select
        dc.customer_key,
        dc.loyalty_tier,
        dc.acquisiton_channel,
        sum(fs.line_total) as total_customer_spend
    from dim_customer dc
    left join fact_sales fs on dc.customer_key = fs.customer_key
    group by
        dc.customer_key,
        dc.loyalty_tier,
        dc.acquisiton_channel)

select
    loyalty_tier,
    acquisiton_channel,
    count(customer_key) as total_customers,
    round(avg(total_customer_spend), 2) as avg_customer_ltv
from customer_ltv
group by 1, 2
order by 4;
```

loyalty_tier	acquisiton_channel	total_customers	avg_customer_ltv
Basic	Referral	602	295.75
Basic	In-Store Signup	1,478	303.18
Basic	Social Media Ad	862	306.51
Basic	Online Signup	1,258	307.95
Silver	Social Media Ad	212	1,253.62
Silver	Online Signup	318	1,261.56
Silver	Referral	164	1,283.44
Silver	In-Store Signup	386	1,286.39
Gold	In-Store Signup	182	2,943.31
Gold	Referral	94	2,965.53
Gold	Online Signup	150	3,051.54
Gold	Social Media Ad	114	3,132.61
Platinum	In-Store Signup	69	7,880.43
Platinum	Online Signup	43	8,386.78
Platinum	Referral	35	8,426.82

Products With The Best Profit Margin

```
select
    dp.product_id,
    dp.product_name,
    dp.brand,
    sum(fs.quantity) as total_amount_sold,
    round(sum(fs.line_total - (fs.quantity * dp.unit_cost)), 2) as total_profit,
    round((sum(fs.line_total - (fs.quantity * dp.unit_cost)) / nullif(sum(fs.line_total), 0)) * 100, 2) as profit_margin
from dim_product dp
join fact_sales fs on fs.product_key = dp.product_key
group by 1, 2, 3
order by 6 desc
limit 10
```

product_id	product_name	brand	total_amount_sold	total_profit	profit_margin
P0185	Pure Face Wash	Fieldstone	231	4,644.57	70.31
P0209	Retinol Beard Balm	Anchor & Co.	404	4,439.24	70.18
P0135	Aloe Eau de Toilette	Amber & Oak	588	55,803.56	70.16
P0027	Green Tea Overnight Mask	Northbloom	655	6,609.36	70.12
P0089	Radiant Lip Liner	Bare Canvas	657	7,758.68	70.09
P0058	Rose Quartz Eyeshadow Palette - Blush Petal	Velvet Noir	562	18,846.45	70.06
P0007	Everyday Serum	Cedar & Rain	637	44,792.05	70.05
P0036	Golden Hour Serum	Hydralux	419	10,131.82	70.03
P0166	Pure Hand Cream	Meadow Bath Co.	997	10,061.49	69.98
P0053	Golden Hour Eyeshadow Palette - Blush Petal	Velvet Noir	530	10,446.88	69.97

Online vs In Store

```
select
    fs.channel,
    count(distinct fs.order_id) as total_orders,
    sum(fs.quantity) as total_units,
    round(sum(fs.line_total), 2) as total_revenue,
    round(sum(fs.line_total) / count(distinct fs.order_id), 2) as avg_order_value
from fact_sales fs
group by 1
order by 4 desc
```

channel	total_orders	total_units	total_revenue	avg_order_value
In-Store	34,148	82,319	3,550,150.86	103.96
Online	18,234	44,155	1,898,164.98	104.1

Best Stores of Every Region

```sql
with stores as(
    select
        ds.region,
        ds.store_id,
        ds.store_name,
        ds.store_type,
        count(distinct fs.order_id) as total_orders,
        round(sum(fs.line_total), 2) as total_revenue,
        round(sum(fs.line_total), 2) / nullif(count(distinct fs.order_id), 0) as aov,
        row_number() over(partition by ds.region order by sum(fs.line_total) desc) as region_rnk
    from dim_store ds
    join fact_sales fs on ds.store_key = fs.store_key
    where ds.store_name != 'Online Store'
    group by 1, 2, 3, 4)

select
    region,
    store_name,
    store_type,
    total_orders,
    total_revenue,
    aov as average_order_value
from stores
where region_rnk = 1
order by total_revenue desc;
```

region	store_name	store_type	total_orders	total_revenue	average_order_value
Northeast	Vaelora Beauty - New York City Outlet	Outlet	3,836	395,978.54	103.2269395203
West	Vaelora Beauty - Los Angeles Mall II	Mall	2,023	217,978.13	107.7499406822
South	Vaelora Beauty - Charlotte Outlet	Outlet	1,721	189,712.18	110.2336897153
Midwest	Vaelora Beauty - Ann Arbor Outlet II	Outlet	1,648	181,758.24	110.2901941748

Acquisition Channel Performance

```sql
select
    dc.acquisiton_channel,
    count(distinct fs.customer_key) as unique_customers,
    count(distinct fs.order_id) as total_orders,
    sum(fs.quantity) as total_sold,
    round(sum(fs.line_total), 2) as total_revenue
from fact_sales fs
join dim_customer dc on fs.customer_key = dc.customer_key
group by 1
order by 2 desc;
```

acquisiton_channel	unique_customers	total_orders	total_sold	total_revenue
In-Store Signup	1,762	18,445	44,500	1,917,054.25
Online Signup	1,454	14,540	35,132	1,509,937.23
Social Media Ad	1,018	10,647	25,553	1,106,122.79
Referral	736	8,750	21,289	915,201.57

Campaign Performance and Top-Selling Product per Campaign

```sql
with campaign_totals as (
    select
        c.campaign_name,
        count(distinct fs.customer_key) as unique_customers,
        count(distinct fs.order_id) as total_orders,
        sum(fs.quantity) as total_sold,
        round(sum(fs.line_total), 2) as total_revenue
    from fact_sales fs
    join dim_campaign c on fs.campaign_key = c.campaign_key
    group by c.campaign_name
),
top_products as (
    select
        c.campaign_name,
        dp.product_key,
        dp.product_name,
        dp.brand,
        row_number() over(partition by c.campaign_name order by sum(fs.quantity) desc, dp.product_key) as rnk
    from fact_sales fs
    join dim_campaign c on fs.campaign_key = c.campaign_key
    join dim_product dp on fs.product_key = dp.product_key
    group by c.campaign_name, dp.product_key, dp.product_name, dp.brand
)
select
    ct.campaign_name,
    tp.product_name || ' (' || tp.brand || ')' as top_selling_product,
    ct.unique_customers,
    ct.total_orders,
    ct.total_sold sold,
    ct.total_revenue
from campaign_totals ct
join top_products tp on ct.campaign_name = tp.campaign_name
where tp.rnk = 1 and ct.campaign_name != 'No Campaign'
order by ct.total_revenue desc;
```

A-Z campaign_name	A-Z top_selling_product	123 unique_customers	123 total_orders	123 sold	123 total_revenue
12 Days of Beauty 2025	Matte Eye Gel (Northbloom)	1,215	1,508	3,541	156,205.32
12 Days of Beauty 2024	Matte Eye Gel (Northbloom)	866	1,377	3,230	139,834.24
Black Friday Beauty Blowout 2025	Aloe Eye Gel (Hydralux)	809	958	2,327	103,037.18
Black Friday Beauty Blowout 2024	Sea Mineral Sunscreen SPF 50 (Northbloom)	628	871	2,206	86,787.32
Mother's Day Beauty Gifts 2025	Aloe Eye Gel (Hydralux)	473	534	1,247	54,642.57
Spring Refresh Sale 2025	Oat Milk Eye Cream (Hydralux)	420	497	1,201	52,901.37
Mother's Day Beauty Gifts 2024	Vitamin C Sunscreen SPF 30 (Cedar & Rain)	316	502	1,226	52,786.61
Halloween Glam Event 2025	Vitamin C Sunscreen SPF 30 (Cedar & Rain)	434	473	1,130	51,368.96
Spring Refresh Sale 2024	Vitamin C Sunscreen SPF 30 (Cedar & Rain)	225	457	1,173	51,020.17
New Year Skin Reset 2025	Weightless Eye Gel (Hydralux)	409	472	1,150	49,056.01
Valentine's Gift Edit 2025	Pure Foundation (Velvet Noir)	404	464	1,098	47,855.38
Back-to-Beauty Event 2024	Oat Milk Eye Cream (Hydralux)	337	431	1,072	47,817.54
Summer Skin Essentials 2024	Aloe Eye Gel (Hydralux)	329	472	1,168	47,572.84
Halloween Glam Event 2024	Overnight Sunscreen SPF 30 (Hydralux)	354	445	1,097	46,969.13
Back-to-Beauty Event 2025	Oat Milk Eye Cream (Hydralux)	405	439	1,109	45,196.14
Summer Skin Essentials 2025	Matte Eye Gel (Northbloom)	436	486	1,104	44,632.82
Mid-Year Clearance 2025	Aloe Eye Gel (Hydralux)	395	440	1,014	43,839.79
Valentine's Gift Edit 2024	Vitamin C Sunscreen SPF 30 (Cedar & Rain)	143	386	925	40,556.55
Mid-Year Clearance 2024	Weightless Eye Gel (Hydralux)	334	423	951	39,701.15
New Year Skin Reset 2024	Aloe Eye Gel (Hydralux)	55	416	954	39,041.16
Fall Skincare Refresh 2024	Vitamin C Sunscreen SPF 30 (Cedar & Rain)	280	329	784	32,608.34
Fall Skincare Refresh 2025	Vitamin C Sunscreen SPF 30 (Cedar & Rain)	305	323	794	31,617.43
Year-End Clearance 2025	Overnight Sunscreen SPF 30 (Hydralux)	142	152	384	16,294.81
Year-End Clearance 2024	Vitamin C Sunscreen SPF 30 (Cedar & Rain)	129	138	347	13,467.13

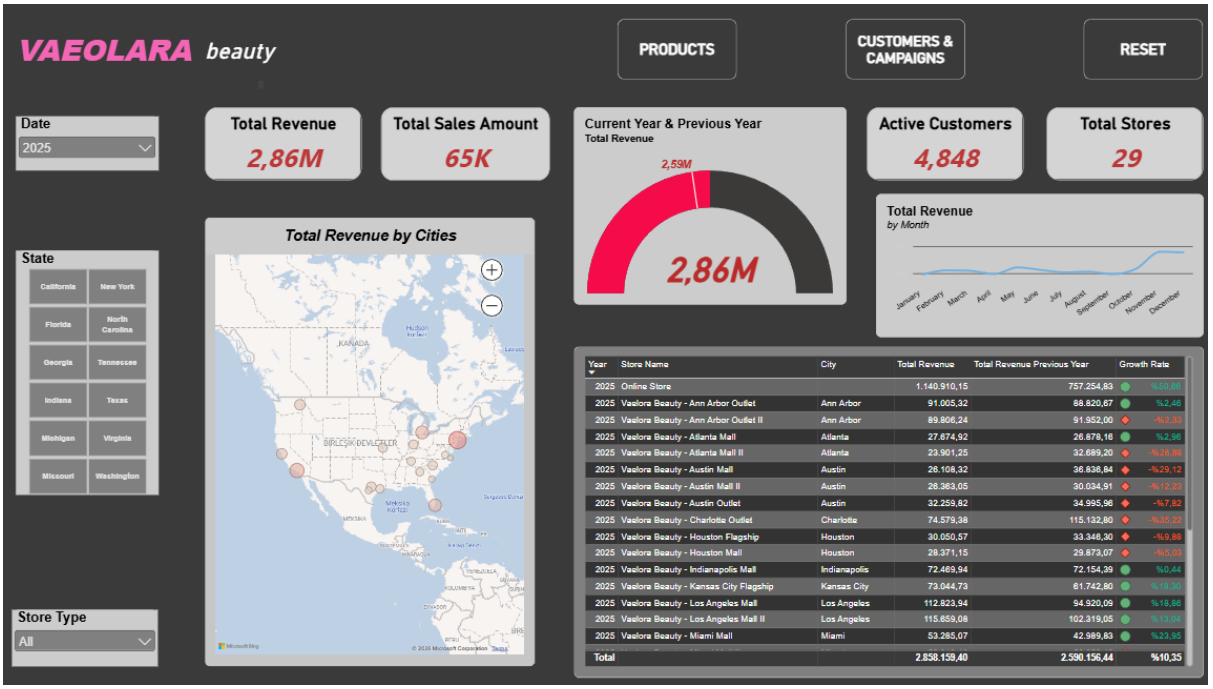
Loyalty Tier Performance

```sql
select
    dc.loyalty_tier,
    count(distinct dc.customer_key) as unique_customers,
    sum(fs.quantity) as total_units,
    round(sum(fs.line_total), 2) as total_revenue,
    round(sum(fs.line_total) / count(distinct fs.order_id), 2) as avg_order_value,
    round(sum(fs.line_total) / count(distinct dc.customer_key), 2) as avg_revenue_per_customer,
    round(count(distinct fs.order_id)::numeric / count(distinct dc.customer_key), 2) as avg_orders_per_customer
from dim_customer dc
left join fact_sales fs on dc.customer_key = fs.customer_key
group by 1
order by 6 desc;
```

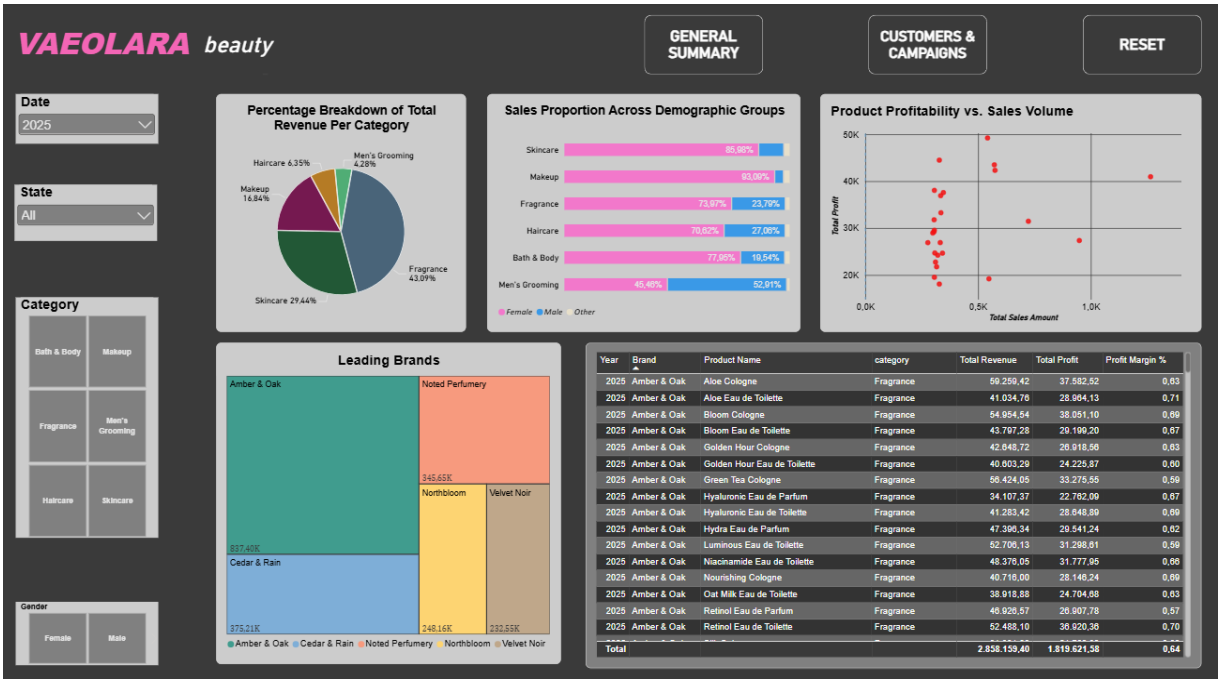
A-Z loyalty_tier	123 unique_customers	123 total_units	123 total_revenue	123 avg_order_value	123 avg_revenue_per_customer	123 avg_orders_per_customer
Platinum	180	33,814	1,480,564.92	106.22	8,225.36	77.44
Gold	540	37,095	1,629,290.72	107.87	3,017.21	27.97
Silver	1,080	31,081	1,373,973.18	108.9	1,272.2	11.68
Basic	4,200	24,484	964,487.02	89.95	229.64	2.55

Power BI Report

SUMMARY PAGE



PRODUCTS PAGE



CUSTOMERS & CAMPAIGN PAGE



The report was created through a PostgreSQL connection. It has three pages. The summary page provides general information about the stores, such as store locations, the number of active customers, total stores, and a comparison of revenue with the previous year. At the bottom right of the page, there is a table that shows information about all the stores. The user can filter the table and overall information using the filter buttons on the left side, or can directly click on a bubble in the map or a store in the table. The table shows the growth rate of the stores compared to the previous year. Stores with a positive growth rate are highlighted in green, while stores with a negative growth rate are highlighted in red. The bubbles on the map also get bigger with higher revenue.

The second page is the products page. As the name suggests, this page consists of information about the products on the shelves or on the website. There is a new 'Categories' filter on this page that filters products according to their relevant categories. One of the most insightful elements of this page is in the bottom right: the Product Profitability vs. Sales Volume scatterplot. As the

name suggests, this scatterplot shows the user products that bring a huge amount of profit despite low sales volumes, the exact opposite, or products that bring huge profits due to a great amount of sales. If you hover over the dots on the graphic, you can see the name, total sales amount, profit margin, and category of the relevant item. This page also has a graphic that shows the distribution of genders by category. Users can see that women dominate overall sales in almost all categories. Revenue distribution by category is also shown in a pie chart. The leading brands graphic shows the overall leading brands in revenue, but users can filter this information using the filters on the left or by interacting with the graphics. For example, selecting haircare from the pie chart will show the leading brands in haircare. Two different filters can be chosen, such as the leading brands of haircare in 2024. The table associated with this graphic shows product information from all brands. The most important metric in this table is the profit margin, which is a financial ratio measuring the percentage of revenue a business keeps as profit after paying its costs.

The last page shows information about customers and campaigns. It displays details about demographics, such as how customers are distributed among loyalty tiers, the age and gender distribution of active customers, the most popular payment methods, and the most popular campaigns. When you hover over a loyalty tier, age bracket, or payment method, you can see deeper information regarding total revenue, sales, and customer count. The tree visual at the bottom right offers a more comprehensive look at these preferences. Like the other two pages, this page can also be filtered through the visuals. For example, applying filters on the bottom-right visual shows the revenue from the fragrance category for male users in the silver loyalty tier whose acquisition channel was a physical store.

Conclusion

This project was developed using synthetic data, meaning it may not fully reflect real-world scenarios. The workflow followed several key stages. First, it went through a data cleaning phase to handle null values, detect inconsistencies, and correct data types. Next, the data was loaded into PostgreSQL, where it was structured into a star schema consisting of dimension tables and a fact table. This star schema design was implemented to simplify data relationships, optimize analytical query performance, and ensure efficient data modeling within Power BI.

Using these tables, an exploratory analysis was conducted through SQL queries to better understand the data, uncovering insights such as customer lifetime value by acquisition channel, top-performing stores by state, loyalty tier performance, and the revenue difference between online and in-store channels. Following this, Power BI connected to the PostgreSQL database. Utilizing in-app calculation tools and DAX formulas, a comprehensive three-page Power BI report was built. This allows stakeholders to track KPIs and make informed decisions, providing a highly interactive and practical tool centered around actionable knowledge.